Trabajo Práctico · Unidad 7: NoSQL y Nuevas Formas de Almacenamiento

Food Store — Modelado Documental en MongoDB, Validación `$jsonSchema`, Precisión `Decimal128`, Validación Cruzada y Evaluación Crítica con IA

Cátedra: Bases de Datos II — Tecnicatura Universitaria en Programación (UTN FRM)

Integrantes: Tiziano Valentini y Juan Martin Garcia

Repositorio Institucional: `Food-Store-Bd-Valentini-Garcia` (rama: `trabajos-practicos-nosql`)

Base Relacional de Origen: `food_store_dev` (PostgreSQL 18)

Base Documental de Destino: `food_store_nosql` (MongoDB 8.0 / `mongosh`)

Parte 1 · Modelo Documental, Validador `$jsonSchema` y Migración con `Decimal128`

1. Tablas y Claves de PostgreSQL Intervinientes (Consigna 1)

El subconjunto seleccionado para el modelado documental corresponde al núcleo transaccional de ventas de Food Store: **Pedidos y Detalles de Pedido**, interactuando directamente con Clientes, Productos y Categorías. En el esquema relacional (`TP1_FoodStore/schema.sql` y `TP5_Indices_Vistas/soft_delete.sql`), las tablas y restricciones que intervienen son:

1. `Pedido`:

- **Clave Primaria (PK):** `id_pedido` (`BIGINT GENERATED ALWAYS AS IDENTITY`).
- **Clave Foránea (FK):** `id_cliente` → `Cliente(id_cliente)` con regla `ON DELETE RESTRICT` (impide eliminar clientes con historial de comprobantes asociados).
- **Atributos:** `fecha_hora` (`TIMESTAMPTZ`), `forma_pago` (`forma_pago_enum`: `'EFECTIVO'`, `'TARJETA'`, `'TRANSFERENCIA'`, `'BILLETERA_DIGITAL'`), `estado` (`VARCHAR(20)`: `'ENTREGADO'`, `'EN_PREPARACION'`, `'PENDIENTE'`, `'CANCELADO'`), `eliminado` (`BOOLEAN NOT NULL DEFAULT FALSE` para soft delete).

1. `Detalle_Pedido` (Entidad asociativa N:M):

- **Clave Primaria Compuesta (PK):** `(id_pedido, id_producto)`.
- **Claves Foráneas (FK):** `id_pedido` → `Pedido(id_pedido)` con `ON DELETE CASCADE` (la anulación física del comprobante arrastra sus líneas), e `id_producto` → `Producto(id_producto)` con `ON DELETE RESTRICT`.
- **Restricciones de Dominio:** `chk_detalle_cantidad_positiva` (`CHECK (cantidad > 0)`) y `chk_detalle_precio_no_negativo` (`CHECK (precio_unitario >= 0)`).
- **Tipado Monetario Exacto:** `precio_unitario` definido como `NUMERIC(10,2) NOT NULL`.
- **Auditoría y Vigencia:** `eliminado` (`BOOLEAN DEFAULT FALSE`) y `stock_descontado` (`BOOLEAN DEFAULT FALSE`).

1. `Cliente`:

- **Clave Primaria (PK):** `id_cliente` (`BIGINT`).
- **Clave Candidata:** `correo` (`VARCHAR(150)`) con índice único parcial `uq_cliente_correo_vigente` `WHERE eliminado = FALSE`).
- **Atributos:** `nombre`, `telefono`, `eliminado`, `created_at`.

1. `Producto` y `Categoria`:

- **Producto:** PK `id_producto`, FK `id_categoria` → `Categoria(id_categoria) ON DELETE RESTRICT`, `nombre_producto`, `precio_actual` (`NUMERIC(10,2)`), `stock` (`INT`), `activo` (`BOOLEAN`).
- **Categoria:** PK `id_categoria`, `nombre_categoria`, `descripcion`.

---

2. Esquema Formal de la Colección Documental `pedidos` y Tipado `Decimal128`

En MongoDB, la colección adopta el patrón de diseño basado en la **Raíz del Agregado (Aggregate Root)** de Domain-Driven Design (DDD). Cada documento almacena de forma autosuficiente la cabecera, la instantánea del cliente y el arreglo de líneas.

Para evitar los errores de redondeo de coma flotante binaria IEEE 754 inherentes al tipo `double` nativo de JavaScript, **todos los campos monetarios (`precio_unitario`, `subtotal` y `total_pedido`) se almacenan estrictamente en tipo BSON `Decimal128` (`NumberDecimal("...")`, `bsonType: "decimal"`)**, garantizando representación decimal exacta en base 10 equivalente al `NUMERIC(10,2)` de PostgreSQL:

```
{
  "_id": "ObjectId",
  "id_pedido": "Int32 / Int64 (Identificador conservado de PostgreSQL, minimum: 1, índice UNIQUE)",
  "fecha_hora": "ISODate (Timestamp de la transacción, bsonType: 'date')",
  "forma_pago": "String enum ('EFECTIVO' | 'TARJETA' | 'TRANSFERENCIA' | 'BILLETERA_DIGITAL')",
  "estado": "String enum ('ENTREGADO' | 'EN_PREPARACION' | 'PENDIENTE' | 'CANCELADO')",
  "eliminado": "Boolean (Bandera de soft delete, bsonType: 'bool')",
  "cliente": {
    "id_cliente": "Int32 / Int64 (Referencia relacional de origen, minimum: 1)",
    "nombre": "String (minLength: 1)",
    "email": "String (minLength: 3)"
  },
  "lineas": [
    {
      "id_producto": "Int32 / Int64 (minimum: 1)",
      "nombre_producto": "String (minLength: 1)",
      "categoria": "String (Desnormalizada desde Categoria, minLength: 1)",
      "cantidad": "Int32 / Int64 (Entero positivo, minimum: 1)",
      "precio_unitario": "Decimal128 / NumberDecimal (bsonType: 'decimal', precio histórico pactado)",
      "subtotal": "Decimal128 / NumberDecimal (bsonType: 'decimal', cantidad * precio_unitario)"
    }
  ],
  "total_pedido": "Decimal128 / NumberDecimal (bsonType: 'decimal', suma exacta de subtotales)"
}
```

---

3. Creación de la Colección con Validador Formal `$jsonSchema`

Dado que MongoDB no impone restricciones de esquema por defecto si la colección se crea implícitamente, se incorporó en `consultas_mongodb.js` la creación explícita de la colección `pedidos` con un **validador formal `$jsonSchema`** con nivel `validationLevel: "strict"` y acción `validationAction: "error"`. El validador exige todos los campos obligatorios (`required`), restringe los dominios de `estado` y `forma_pago` mediante `enum`, exige que `lineas` sea un arreglo no vacío (`minItems: 1`) con cantidades positivas (`minimum: 1`), e impone `bsonType: "decimal"` (`Decimal128`) sobre `precio_unitario`, `subtotal` y `total_pedido`:

```
db.createCollection("pedidos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      title: "Validador de esquema para la coleccion pedidos de Food Store",
      required: [
        "id_pedido", "fecha_hora", "forma_pago", "estado",
        "eliminado", "cliente", "lineas", "total_pedido"
      ],
      properties: {
        id_pedido: { bsonType: ["int", "long"], minimum: 1 },
        fecha_hora: { bsonType: "date" },
        forma_pago: { enum: ["EFECTIVO", "TARJETA", "TRANSFERENCIA", "BILLETERA_DIGITAL"] },
        estado: { enum: ["ENTREGADO", "EN_PREPARACION", "PENDIENTE", "CANCELADO"] },
        eliminado: { bsonType: "bool" },
        cliente: {
          bsonType: "object",
          required: ["id_cliente", "nombre", "email"],
          properties: {
            id_cliente: { bsonType: ["int", "long"], minimum: 1 },
            nombre: { bsonType: "string", minLength: 1 },
            email: { bsonType: "string", minLength: 3 }
          }
        },
        lineas: {
          bsonType: "array",
          minItems: 1,
          items: {
            bsonType: "object",
            required: [
              "id_producto", "nombre_producto", "categoria",
              "cantidad", "precio_unitario", "subtotal"
            ],
            properties: {
              id_producto: { bsonType: ["int", "long"], minimum: 1 },
              nombre_producto: { bsonType: "string", minLength: 1 },
              categoria: { bsonType: "string", minLength: 1 },
              cantidad: { bsonType: ["int", "long"], minimum: 1 },
              precio_unitario: { bsonType: "decimal" },
              subtotal: { bsonType: "decimal" }
            }
          }
        },
        total_pedido: { bsonType: "decimal" }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});
```

---

4. Evidencia Real de Inserción `insertMany()` con `Decimal128` y Prueba del Validador (Consigna 2)

Se migraron los 6 pedidos reales desde la base relacional `food_store_dev`, utilizando `NumberDecimal("...")` en todos los importes (`precio_unitario`, `subtotal` y `total_pedido`), y a continuación se probó insertar un documento inválido con importes en `double` (`1050.50`) para verificar el rechazo activo del motor.

Salida real de consola (`mongosh`):

```
================================================================================
 >>> SELECCIONANDO BASE DE DATOS: food_store_nosql
================================================================================

 >>> CREACIÓN DE COLECCIÓN 'pedidos' CON VALIDADOR $jsonSchema:
{
  ok: 1
}

 >>> EVIDENCIA DE INSERCIÓN (insertMany con Decimal128):
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6af3a5e587c7e71e6bb800df'),
    '1': ObjectId('6af3a5e587c7e71e6bb800e0'),
    '2': ObjectId('6af3a5e587c7e71e6bb800e1'),
    '3': ObjectId('6af3a5e587c7e71e6bb800e2'),
    '4': ObjectId('6af3a5e587c7e71e6bb800e3'),
    '5': ObjectId('6af3a5e587c7e71e6bb800e4')
  }
}
 >>> Total documentos insertados: 6

================================================================================
 PRUEBA DE VALIDADOR $jsonSchema: Intento de inserción con importe en double
================================================================================
 >>> RECHAZO ESPERADO DEL MOTOR MONGODB (Document failed validation):
Codigo de error: 121 (DocumentValidationFailure)
Mensaje: Document failed validation
{
  failingDocumentId: ObjectId('6af3a5e587c7e71e6bb800e5'),
  details: {
    operatorName: '$jsonSchema',
    title: 'Validador de esquema para la coleccion pedidos de Food Store',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'properties',
        propertiesNotSatisfied: [
          {
            propertyName: 'lineas',
            description: 'Arreglo no vacio de renglones del pedido (minimo 1 item).',
            details: [
              {
                operatorName: 'items',
                reason: 'At least one item did not match the sub-schema',
                itemIndex: 0,
                details: [
                  {
                    operatorName: 'properties',
                    propertiesNotSatisfied: [
                      {
                        propertyName: 'precio_unitario',
                        description: 'Precio unitario historico pactado en BSON Decimal128 (NumberDecimal).',
                        details: [
                          {
                            operatorName: 'bsonType',
                            specifiedAs: { bsonType: 'decimal' },
                            reason: 'type did not match',
                            consideredValue: 1050.5,
                            consideredType: 'double'
                          }
                        ]
                      },
                      {
```

```
                    propertyName: 'subtotal',
                    description: 'Subtotal de la linea (cantidad * precio_unitario) en BSON Decimal128.',
                    details: [
                      {
                        operatorName: 'bsonType',
                        specifiedAs: { bsonType: 'decimal' },
                        reason: 'type did not match',
                        consideredValue: 1050.5,
                        consideredType: 'double'
                      }
                    ]
                  }
                ]
              }
            ]
          }
        ]
      },
      {
        propertyName: 'total_pedido',
        description: 'Importe total del pedido en BSON Decimal128 (NumberDecimal).',
        details: [
          {
            operatorName: 'bsonType',
            specifiedAs: { bsonType: 'decimal' },
            reason: 'type did not match',
            consideredValue: 1050.5,
            consideredType: 'double'
          }
        ]
      }
    ]
  }
}
```

---

5. Reglas de Integridad Conservadas y Justificación de Embedding (Consigna 3)

A. Reglas de Integridad Conservadas:

1. **Inmutabilidad del precio histórico de venta y precisión decimal (`Decimal128`):** En el esquema de PostgreSQL, la tabla `Detalle_Pedido` almacena `precio_unitario` como `NUMERIC(10,2)` , independizándolo de las fluctuaciones en `Producto.precio_actual` . Durante la migración documental se preservó este valor histórico exacto en cada ítem de `lineas` utilizando `NumberDecimal("...")` (`Decimal128`) y se blindó mediante `bsonType: "decimal"` en el validador `$jsonSchema` . Esto garantiza tanto la exactitud contable en base 10 como que eventuales aumentos en el catálogo no alteren retroactivamente los comprobantes emitidos.
2. **Consistencia de dominio, cardinalidad mínima y cantidades positivas:** Se conservó la regla de negocio donde `total_pedido = SUM(cantidad * precio_unitario)` en `Decimal128` , además de hacer cumplir en el propio motor documental que todo pedido tenga al menos un ítem (`minItems: 1`), que cada línea tenga cantidad estrictamente positiva (`minimum: 1` , equivalente a `chk_detalle_cantidad_positiva`) y que los estados y medios de pago pertenezcan a los conjuntos válidos (`enum`).

B. Justificación Técnica de la Estrategia de Embebido (Embedding vs Referencing):

- **Líneas del pedido embebidas (`lineas: [...]`):** Una línea de pedido no posee existencia conceptual independiente; su ciclo de vida está 100% subordinado al pedido. En sistemas relacionales, su separación

en `Detalle_Pedido` obedece a la Primera Forma Normal (1FN), obligando a ejecutar múltiples `JOIN` en cada lectura. En MongoDB, el embebido garantiza **localidad de referencia espacial en disco** (los datos se leen contiguos en una única operación de I/O de WiredTiger) y **atomicidad a nivel de documento único** (*single-document atomicity*), eliminando la necesidad de transacciones multi-documento para crear o consultar un pedido con todos sus ítems.

- **Datos esenciales del cliente embebidos (`cliente: { id_cliente, nombre, email }`):** Se adoptó el patrón de *Desnormalización Deliberada por Snapshot*. Un comprobante de compra debe reflejar los datos del cliente al instante exacto de la operación. Si en el futuro el cliente actualiza su nombre o correo en la colección maestra, el pedido histórico conserva inalterada la foto con la que fue emitido.

Parte 2 · Consultas y Verificación Cruzada

1. Ejecución de Consultas en MongoDB (`mongosh` — Consigna 4)

Consulta (a): Mostrar todos los pedidos con identificador y estado

```
db.pedidos.find(
  {},
  { id_pedido: 1, estado: 1, forma_pago: 1, total_pedido: 1, _id: 0 }
).sort({ id_pedido: 1 }).toArray();
```

Salida real de consola (`mongosh`):

```
[
  {
    id_pedido: 1,
    forma_pago: 'EFECTIVO',
    estado: 'ENTREGADO',
    total_pedido: Decimal128('17804.34')
  },
  {
    id_pedido: 2,
    forma_pago: 'TARJETA',
    estado: 'ENTREGADO',
    total_pedido: Decimal128('17662.15')
  },
  {
    id_pedido: 3,
    forma_pago: 'TRANSFERENCIA',
    estado: 'EN_PREPARACION',
    total_pedido: Decimal128('12539.38')
  },
  {
    id_pedido: 4,
    forma_pago: 'TARJETA',
    estado: 'ENTREGADO',
    total_pedido: Decimal128('3175.70')
  },
  {
    id_pedido: 5,
    forma_pago: 'BILLETERA_DIGITAL',
    estado: 'ENTREGADO',
    total_pedido: Decimal128('25104.04')
  },
  {
    id_pedido: 6,
    forma_pago: 'TRANSFERENCIA',
    estado: 'PENDIENTE',
    total_pedido: Decimal128('24299.84')
  }
]
Total recuperados consulta (a): 6
```

Consulta (b): Filtrar pedidos por estado existente (`estado: "ENTREGADO"`)

```
db.pedidos.find(
  { estado: "ENTREGADO" },
  { id_pedido: 1, fecha_hora: 1, forma_pago: 1, estado: 1, "cliente.nombre": 1, total_pedido: 1, _id: 0 }
).sort({ id_pedido: 1 }).toArray();
```

Salida real de consola (`mongosh`):

```
[
  {
    id_pedido: 1,
    fecha_hora: ISODate('2026-09-21T22:22:22.225Z'),
    forma_pago: 'EFECTIVO',
    estado: 'ENTREGADO',
    cliente: { nombre: 'Ana Gómez' },
    total_pedido: Decimal128('17804.34')
  },
  {
    id_pedido: 2,
    fecha_hora: ISODate('2026-09-26T22:22:22.225Z'),
    forma_pago: 'TARJETA',
    estado: 'ENTREGADO',
    cliente: { nombre: 'Luis Paz' },
    total_pedido: Decimal128('17662.15')
  },
  {
    id_pedido: 4,
    fecha_hora: ISODate('2025-01-05T17:38:06.391Z'),
    forma_pago: 'TARJETA',
    estado: 'ENTREGADO',
    cliente: { nombre: 'Cliente 7650' },
    total_pedido: Decimal128('3175.70')
  },
  {
    id_pedido: 5,
    fecha_hora: ISODate('2026-08-29T11:38:48.485Z'),
    forma_pago: 'BILLETERA_DIGITAL',
    estado: 'ENTREGADO',
    cliente: { nombre: 'Cliente 19190' },
    total_pedido: Decimal128('25104.04')
  }
]
Total recuperados consulta (b): 4
```

Consulta (c): Buscar pedidos que contengan un producto determinado (`id_producto = 3` , "Coca 1.5L")

```
db.pedidos.find(
  { "lineas.id_producto": 3 },
  {
    id_pedido: 1,
    fecha_hora: 1,
    estado: 1,
    "cliente.nombre": 1,
    lineas: { $elemMatch: { id_producto: 3 } },
    total_pedido: 1,
    _id: 0
  }
).sort({ id_pedido: 1 }).toArray();
```

Salida real de consola (`mongosh`):

```
[
  {
    id_pedido: 1,
    fecha_hora: ISODate('2026-09-21T22:22:22.225Z'),
    estado: 'ENTREGADO',
    cliente: { nombre: 'Ana Gómez' },
    total_pedido: Decimal128('17804.34'),
    lineas: [
      {
        id_producto: 3,
        nombre_producto: 'Coca 1.5L',
        categoria: 'Bebidas',
        cantidad: 1,
        precio_unitario: Decimal128('800.00'),
        subtotal: Decimal128('800.00')
      }
    ]
  },
  {
    id_pedido: 3,
    fecha_hora: ISODate('2026-09-30T22:22:22.225Z'),
    estado: 'EN_PREPARACION',
    cliente: { nombre: 'Marta Ruiz' },
    total_pedido: Decimal128('12539.38'),
    lineas: [
      {
        id_producto: 3,
        nombre_producto: 'Coca 1.5L',
        categoria: 'Bebidas',
        cantidad: 4,
        precio_unitario: Decimal128('800.00'),
        subtotal: Decimal128('3200.00')
      }
    ]
  }
]
Total recuperados consulta (c): 2
```

Consulta (d): Buscar pedidos con total superior a un importe justificado (`total_pedido > NumberDecimal("15000.00")`)

```
db.pedidos.find(
  { total_pedido: { $gt: NumberDecimal("15000.00") } },
  { id_pedido: 1, estado: 1, "cliente.nombre": 1, total_pedido: 1, _id: 0 }
).sort({ total_pedido: -1 }).toArray();
```

Salida real de consola (`mongosh`):

```
[
  {
    id_pedido: 5,
    estado: 'ENTREGADO',
    cliente: { nombre: 'Cliente 19190' },
    total_pedido: Decimal128('25104.04')
  },
  {
    id_pedido: 6,
    estado: 'PENDIENTE',
    cliente: { nombre: 'Cliente 14469' },
    total_pedido: Decimal128('24299.84')
  },
  {
    id_pedido: 1,
    estado: 'ENTREGADO',
    cliente: { nombre: 'Ana Gómez' },
    total_pedido: Decimal128('17804.34')
  },
  {
    id_pedido: 2,
    estado: 'ENTREGADO',
    cliente: { nombre: 'Luis Paz' },
    total_pedido: Decimal128('17662.15')
  }
]
Total recuperados consulta (d): 4
```

Justificación del Umbral de Importe:

El umbral seleccionado fue de `NumberDecimal("15000.00")` ($15.000,00). En el conjunto muestral de Food Store, los pedidos presentan importes entre $3.175,70 y $25.104,04, con un ticket promedio de **$16.764,24**. El umbral de $15.000,00 discrimina los pedidos de consumo individual o colación rápida (como el Pedido 4 por $3.175,70 o el Pedido 3 por $12.539,38) de aquellos correspondientes a **Compras Mayoristas o Eventos Gastronómicos** (Pedidos 5, 6, 1 y 2).

---

2. Consultas Equivalentes en PostgreSQL (`psql` — Consigna 5)

Equivalente a Consigna 4 (b): Filtro por estado `'ENTREGADO'`

```
SELECT
    p.id_pedido, p.fecha_hora, p.forma_pago, p.estado, c.nombre AS cliente,
    round(SUM(dp.cantidad * dp.precio_unitario)::numeric, 2) AS total_pedido
FROM Pedido p
JOIN Cliente c ON p.id_cliente = c.id_cliente
JOIN Detalle_Pedido dp ON p.id_pedido = dp.id_pedido
WHERE p.estado = 'ENTREGADO'
  AND p.id_pedido BETWEEN 1 AND 6
GROUP BY p.id_pedido, p.fecha_hora, p.forma_pago, p.estado, c.nombre
ORDER BY p.id_pedido;
```

Salida real de consola (`psql`):

```
 id_pedido |          fecha_hora           |    forma_pago     |  estado   |    cliente    | total_pedido
-----------+-------------------------------+-------------------+-----------+---------------+--------------
         1 | 2026-09-21 22:22:22.22525-03  | EFECTIVO          | ENTREGADO | Ana Gómez     |     17804.34
         2 | 2026-09-26 22:22:22.22525-03  | TARJETA           | ENTREGADO | Luis Paz      |     17662.15
         4 | 2025-01-05 14:38:06.391101-03 | TARJETA           | ENTREGADO | Cliente 7650  |      3175.70
         5 | 2026-08-29 08:38:48.485626-03 | BILLETERA_DIGITAL | ENTREGADO | Cliente 19190 |     25104.04
(4 filas)
```

Equivalente a Consigna 4 (c): Pedidos con `id_producto = 3` ('Coca 1.5L')

```
SELECT DISTINCT
    p.id_pedido, p.fecha_hora, p.estado, c.nombre AS cliente,
    dp.id_producto, pr.nombre_producto, dp.cantidad, dp.precio_unitario,
    round((dp.cantidad * dp.precio_unitario)::numeric, 2) AS subtotal
FROM Pedido p
JOIN Cliente c ON p.id_cliente = c.id_cliente
JOIN Detalle_Pedido dp ON p.id_pedido = dp.id_pedido
JOIN Producto pr ON dp.id_producto = pr.id_producto
WHERE dp.id_producto = 3
  AND p.id_pedido BETWEEN 1 AND 6
ORDER BY p.id_pedido;
```

Salida real de consola (`psql`):

```
 id_pedido |          fecha_hora           |     estado     |  cliente   | id_producto | nombre_producto |
cantidad | precio_unitario | subtotal
-----------+-------------------------------+----------------+------------+-------------+-----------------+-----
-----+-----------------+----------
         1 | 2026-09-21 22:22:22.22525-03 | ENTREGADO      | Ana Gómez  |           3 | Coca 1.5L       |
1 |          800.00 |   800.00
         3 | 2026-09-30 22:22:22.22525-03 | EN_PREPARACION | Marta Ruiz |           3 | Coca 1.5L       |
4 |          800.00 |  3200.00
(2 filas)
```

---

3. Tabla de Verificación Cruzada y Análisis Técnico

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs
Estado seleccionado (`ENTREGADO`)	**4**	**4**	**Sí** (IDs: 1, 2, 4, 5)
Producto seleccionado (`id_producto = 3`)	**2**	**2**	**Sí** (IDs: 1, 3)

Análisis de Correspondencia:

No se detectaron discrepancias entre ambos motores ni en los conteos ni en los importes decimales (`NUMERIC(10,2)` vs `Decimal128`).

- En la consulta por **Estado**, ambos motores identificaron unívocamente los 4 pedidos entregados (`1, 2, 4, 5`) con idénticos valores de cabecera y totales (`17804.34` , `17662.15` , `3175.70` y `25104.04`).
- En la consulta por **Producto**, en PostgreSQL fue necesario ejecutar un `JOIN` entre `Pedido` , `Cliente` , `Detalle_Pedido` y `Producto` . En contraste, en MongoDB la consulta filtró directamente el arreglo embebido mediante el índice multikey `"lineas.id_producto"` y proyectó la línea coincidente con `$elemMatch` .

Parte 3 · Evaluación de la Tecnología e Inteligencia Artificial

1. Prompt de Consulta Enviado a la IA (Consigna 6)

```
"Actuá como Arquitecto de Datos Senior en bases de datos relacionales y NoSQL.
Necesito diseñar la colección documental de Pedidos y Detalles de Pedido en MongoDB a partir de
nuestro esquema PostgreSQL de Food Store, migrar 6 pedidos reales, escribir las consultas de
búsqueda y agregación, y comparar PostgreSQL vs MongoDB para nuestro caso real:
1. Modelo actual: Tablas normalizadas (Pedido, Detalle_Pedido, Cliente, Producto, Categoria)
   con claves foráneas, importes en NUMERIC(10,2), triggers de stock y borrado lógico ('eliminado').
2. Patrón de acceso: Consulta frecuente de pedidos completos (cabecera + cliente + líneas con
   subtotales y categoría) y reportes de facturación por categoría.
3. Requisitos de consistencia: Cero tolerancia a desajustes de stock, inmutabilidad y precisión
   exacta en precios históricos y totales facturados.
Entregá el diseño del documento, script de carga y consultas en mongosh, y tu evaluación
comparativa de pros, contras, rendimiento y arquitectura recomendada."
```

---

2. Resumen Técnico de la Respuesta Recibida de la IA (Consigna 6)

Antes de realizar el contraste crítico, se presenta el resumen técnico detallado de lo que entregó la IA (**OpenCode / LLM**) a lo largo de la interacción (registrado también en `duia.md`):

Eje de la Consulta	Respuesta Recibida de la IA (Resumen Técnico)	Decisión y Corrección Aplicada por el Equipo
1. Estructura documental y creación de la colección	La IA propuso embeber los datos básicos de `cliente` (`id_cliente` , `nombre` , `email`) y el arreglo `lineas` dentro de cada documento de la colección `pedidos` . Sin embargo, **creó la colección de forma implícita invocando directamente `db.pedidos.insertMany(...)` sin definir ningún validador `$jsonSchema`** , argumentando que "*una de las ventajas principales de MongoDB es ser schemaless, por lo que no hace falta declarar esquemas ni restricciones en la base*".	**Aceptado el diseño de embebido; corregida la ausencia de validador.** Se incorporó `db.createCollection("pedidos", { validator: { $jsonSchema: { ... } } })` exigiendo campos obligatorios (`required`), `enum` de estados/pagos, `minItems: 1` en `lineas` y `bsonType: "decimal"` para importes.
2. Representación de importes monetarios	Al generar el JSON de migración desde PostgreSQL (`jsonb_agg`) y el array `pedidosMigrados` en JavaScript, la IA **escribió todos los precios y totales como literales numéricos flotantes de JavaScript (`precio_unitario: 3113.43` , `total_pedido: 17804.34`)**, los cuales `mongosh` serializa en BSON como `double` **(coma flotante binaria IEEE 754 de 64 bits)**.	**Rechazado y corregido a `Decimal128` .** Se reemplazaron todos los importes flotantes por `NumberDecimal("...")` **(`Decimal128`)** tanto en la inserción como en los filtros (`NumberDecimal("15000.00")`), evitando errores de redondeo binario en sumatorias contables.
3. Consultas operativas y Pipeline de Agregación	Para las consultas de la Consigna 4, la IA propuso `db.pedidos.find()` con proyección de campos, `$elemMatch` para filtrar el ítem `id_producto: 3` dentro de `lineas` , y `$gt` para el umbral de importe. Para reportes analíticos por categoría, sugirió el pipeline `db.pedidos.aggregate([{ $match: { estado: "ENTREGADO" } }, { $unwind: "$lineas" }, { $group: { _id: "$lineas.categoria", facturacion: { $sum: "$lineas.subtotal" } } }, { $sort: { facturacion: -1 } }])` .	**Aceptado con ajuste de tipos.** Se conservaron las 4 consultas y el pipeline de agregación, corrigiendo el umbral de la consulta (d) a `NumberDecimal("15000.00")` y verificando que `$sum` sobre `Decimal128` retorne resultados exactos en `Decimal128` .
4. Comparativa arquitectónica	En su evaluación teórica, la IA realizó dos afirmaciones extremas: 1) "*MongoDB no ofrece garantías ACID reales y al ser*	**Contrastado y refutado con pruebas de laboratorio.** Se demostró en consola el rechazo

PostgreSQL vs MongoDB	*schemaless no puede impedir en el motor que se inserten documentos malformados o con tipos incorrectos."* 2) *"Migrar todo el sistema a MongoDB siempre superará en velocidad a PostgreSQL porque elimina los JOINs y las agregaciones documentales son más livianas."*	del validador `$jsonSchema`, la atomicidad de documento único, y se midieron los planes de ejecución (`explain("executionStats")` vs `EXPLAIN ANALYZE`) en lecturas puntuales y agregaciones.

---

3. Contraste Técnico y Lectura Crítica de la IA con Evidencias de Laboratorio (Consigna 7)

A continuación se contrastan las propuestas y afirmaciones de la IA contra la documentación oficial de ambos motores y **se presentan los bloques de salida real de consola obtenidos en nuestro laboratorio** que respaldan cada punto:

Contraste 1: Tipado Monetario (`double` vs `Decimal128`) y Validación de Esquema (`$jsonSchema`)

> **Propuesta y afirmación inicial de la IA:** La IA entregó los importes como números nativos de JavaScript (`double`) y omitió crear un validador para la colección `pedidos`, sosteniendo que *"en MongoDB los números con decimales se manejan directamente con el tipo Number de JS y no se definen esquemas en el motor porque la validación pertenece solo a la capa de aplicación"*.

- **Contraste con la Documentación Oficial y Verificación de Laboratorio:**

Propuesta Incorrecta y Peligrosa para Datos Financieros.

1. De acuerdo con la documentación oficial de MongoDB (*BSON Types — Decimal128 & Schema Validation*), los números nativos de JavaScript se almacenan como BSON `double` (coma flotante binaria IEEE 754 de 64 bits), el cual **no puede representar de forma exacta fracciones decimales cotidianas como `0.1` o `0.2`**, acumulando errores de redondeo en sumatorias de facturación. Para aplicaciones financieras e impositivas, MongoDB provee **`Decimal128` (`NumberDecimal`)**, que representa números decimales en base 10 con 34 dígitos significativos exactos.
2. Asimismo, MongoDB soporta validación nativa en el motor mediante **`$jsonSchema`**. Si no se configura un validador con `bsonType: "decimal"`, cualquier script o microservicio podría insertar importes en `double` o pedidos sin líneas.

- **Evidencia Real de Laboratorio 1.A — Imprecisión de `double` (IEEE 754) vs Exactitud de `Decimal128` en `mongosh`:**

Ejecutamos en `mongosh` una proyección sobre `db.pedidos` comparando la suma en `double` frente a `NumberDecimal` y verificando el tipo BSON real almacenado:

```
db.pedidos.aggregate([
  { $limit: 1 },
  {
    $project: {
      _id: 0,
      suma_double_ieee754: { $add: [0.1, 0.2] },
      suma_decimal128_exacta: { $add: [NumberDecimal("0.10"), NumberDecimal("0.20")] },
      total_pedido_almacenado: "$total_pedido",
      tipo_bson_total: { $type: "$total_pedido" }
    }
  }
]).toArray();
```

Salida real obtenida en consola (`mongosh`):

```
[
  {
    suma_double_ieee754: 0.30000000000000004,
    suma_decimal128_exacta: Decimal128('0.30'),
    total_pedido_almacenado: Decimal128('17804.34'),
    tipo_bson_total: 'decimal'
  }
]
```

Demostración: Mientras que el tipo `double` propuesto originalmente por la IA introduce el residuo binario `0.30000000000000004` , el tipo `Decimal128` implementado por el equipo (`tipo_bson_total: 'decimal'`) preserva el valor exacto `Decimal128('0.30')` y `Decimal128('17804.34')` .

- **Evidencia Real de Laboratorio 1.B — Rechazo en el motor mediante el validador `$jsonSchema` :**

Al intentar insertar un pedido con `precio_unitario: 1050.50` , `subtotal: 1050.50` y `total_pedido: 1050.50` en `double` (tal como sugería la primera versión de la IA), el motor MongoDB abortó la escritura con el código `121` (`DocumentValidationFailure`):

```
Codigo de error: 121 (DocumentValidationFailure)
Mensaje: Document failed validation
{
  failingDocumentId: ObjectId('6af3a5e587c7e71e6bb800e5'),
  details: {
    operatorName: '$jsonSchema',
    title: 'Validador de esquema para la coleccion pedidos de Food Store',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'properties',
        propertiesNotSatisfied: [
          {
            propertyName: 'total_pedido',
            description: 'Importe total del pedido en BSON Decimal128 (NumberDecimal).',
            details: [
              {
                operatorName: 'bsonType',
                specifiedAs: { bsonType: 'decimal' },
                reason: 'type did not match',
                consideredValue: 1050.5,
                consideredType: 'double'
              }
            ]
          }
        ]
      }
    ]
  }
}
```

Contraste 2: Atomicidad en Documento Único y Costo de Lectura de un Pedido Completo (`mongosh` vs `psql`)

> **Afirmación de la IA:** *"MongoDB no ofrece garantías ACID reales en pedidos con múltiples líneas, pero su lectura de un pedido completo es directa porque evita los JOINs de PostgreSQL."*

- **Contraste con la Documentación Oficial y Resultados de Laboratorio:**

Parcialmente Falsa en lo transaccional, Verdadera en el patrón de lectura por clave:

1. Es falso que no exista atomicidad para un pedido con sus líneas: en nuestro diseño documental las líneas (`lineas`) y el snapshot del cliente (`cliente`) están **embebidos dentro del mismo documento**. El motor WiredTiger de MongoDB garantiza **Atomicidad a Nivel de Documento Único (*Single-Document Atomicity*)** (y además soporta transacciones multi-documento ACID desde MongoDB 4.0). La escritura de un pedido con sus 4 líneas y su `total_pedido` ocurre en una única operación atómica sin bloqueos entre tablas.
2. Para verificar empíricamente la diferencia en el costo de lectura de un pedido completo (`id_pedido = 1`), medimos en el laboratorio el plan de ejecución real en ambos motores.

- **Evidencia Real de Laboratorio 2.A — Plan de ejecución en MongoDB (`explain("executionStats")`):**

```
db.pedidos.find({ id_pedido: 1 }).explain("executionStats");
```

Salida real obtenida en consola (`mongosh`):

```
{
  queryPlanner: {
    winningPlan: {
      isCached: false,
      stage: 'EXPRESS_IXSCAN',
      keyPattern: { id_pedido: 1 },
      indexName: 'idx_pedidos_id_pedido_uq'
    }
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 3,
    totalKeysExamined: 1,
    totalDocsExamined: 1
  }
}
```

- **Evidencia Real de Laboratorio 2.B — Plan de ejecución equivalente en PostgreSQL (`EXPLAIN ANALYZE`):**

Para obtener exactamente los mismos datos del Pedido 1 en PostgreSQL (`Pedido` + `Cliente` + `Detalle_Pedido` + `Producto` + `Categoria`), ejecutamos:

```
EXPLAIN ANALYZE
SELECT
    p.id_pedido, p.fecha_hora, p.forma_pago, p.estado,
    c.id_cliente, c.nombre, c.correo,
    pr.id_producto, pr.nombre_producto, cat.nombre_categoria,
    dp.cantidad, dp.precio_unitario,
    round((dp.cantidad * dp.precio_unitario)::numeric, 2) AS subtotal
FROM Pedido p
JOIN Cliente c ON p.id_cliente = c.id_cliente
JOIN Detalle_Pedido dp ON p.id_pedido = dp.id_pedido
JOIN Producto pr ON dp.id_producto = pr.id_producto
JOIN Categoria cat ON pr.id_categoria = cat.id_categoria
WHERE p.id_pedido = 1;
```

Salida real obtenida en consola (`psql`):

```
                                                
```

PostgreSQL.

2. Si bien ambos motores arrojan **exactamente los mismos totales decimales por categoría** (gracias al uso de `Decimal128` en MongoDB y `NUMERIC(10,2)` en PostgreSQL), en MongoDB la etapa `$unwind: "$lineas"` debe **desarmar cada arreglo y multiplicar en memoria RAM un documento virtual por cada renglón** antes de agrupar con `$group`.
3. Más importante aún: el descuento de stock ante ventas concurrentes (`Producto.stock`) requiere actualizar otra entidad distinta al pedido. En PostgreSQL, esto está garantizado por el bloqueo pesimista `SELECT ... FOR UPDATE` dentro del trigger `fn_validar_stock_pedido()` (`TP2_Concurrencia_IA/restricciones.sql`), mientras que en MongoDB obligaría a coordinar transacciones multi-documento entre las colecciones `pedidos` y `productos`, perdiendo la ventaja de escritura en documento único.

- **Evidencia Real de Laboratorio 3.A — Pipeline `$match` + `$unwind` + `$group` con `Decimal128` en `mongosh`:**

```
db.pedidos.aggregate([
  { $match: { estado: "ENTREGADO", eliminado: false } },
  { $unwind: "$lineas" },
  {
    $group: {
      _id: "$lineas.categoria",
      cantidad_lineas: { $sum: 1 },
      unidades_vendidas: { $sum: "$lineas.cantidad" },
      facturacion_categoria: { $sum: "$lineas.subtotal" }
    }
  },
  { $sort: { facturacion_categoria: -1 } }
]).toArray();
```

Salida real obtenida en consola (`mongosh`):

```
[
  {
    _id: 'Almacén',
    cantidad_lineas: 2,
    unidades_vendidas: 6,
    facturacion_categoria: Decimal128('18609.66')
  },
  {
    _id: 'Postres',
    cantidad_lineas: 2,
    unidades_vendidas: 4,
    facturacion_categoria: Decimal128('13168.64')
  },
  {
    _id: 'Panificados',
    cantidad_lineas: 1,
    unidades_vendidas: 4,
    facturacion_categoria: Decimal128('12721.24')
  },
  {
    _id: 'Bebidas',
    cantidad_lineas: 2,
    unidades_vendidas: 4,
    facturacion_categoria: Decimal128('12470.99')
  },
  {
    _id: 'Pizzas',
    cantidad_lineas: 2,
    unidades_vendidas: 3,
    facturacion_categoria: Decimal128('3600.00')
  },
  {
    _id: 'Congelados',
    cantidad_lineas: 1,
    unidades_vendidas: 2,
    facturacion_categoria: Decimal128('2364.26')
  },
  {
    _id: 'Snacks',
    cantidad_lineas: 1,
    unidades_vendidas: 1,
    facturacion_categoria: Decimal128('811.44')
  }
]
```

- **Evidencia Real de Laboratorio 3.B — Consulta Agregada Equivalente en PostgreSQL (`psql`):**

```
SELECT
    cat.nombre_categoria AS categoria,
    COUNT(*) AS cantidad_lineas,
    SUM(dp.cantidad) AS unidades_vendidas,
    round(SUM(dp.cantidad * dp.precio_unitario)::numeric, 2) AS facturacion_categoria
FROM Pedido p
JOIN Detalle_Pedido dp ON p.id_pedido = dp.id_pedido
JOIN Producto pr ON dp.id_producto = pr.id_producto
JOIN Categoria cat ON pr.id_categoria = cat.id_categoria
WHERE p.estado = 'ENTREGADO'
  AND p.eliminado = FALSE
  AND p.id_pedido BETWEEN 1 AND 6
GROUP BY cat.nombre_categoria
ORDER BY facturacion_categoria DESC;
```

Salida real obtenida en consola (`psql`):

```
   categoria  | cantidad_lineas | unidades_vendidas | facturacion_categoria
--------------+-----------------+-------------------+-----------------------
 Almacén      |               2 |                 6 |              18609.66
 Postres      |               2 |                 4 |              13168.64
 Panificados  |               1 |                 4 |              12721.24
 Bebidas      |               2 |                 4 |              12470.99
 Pizzas       |               2 |                 3 |               3600.00
 Congelados   |               1 |                 2 |               2364.26
 Snacks       |               1 |                 1 |                811.44
(7 filas)
```

---

4. Conclusión Arquitectónica Fundamentada (Límite: 200 palabras)

Para el subsistema de pedidos de Food Store, la arquitectura óptima es un **esquema híbrido** (*Polyglot Persistence*). **PostgreSQL** debe consolidarse como la Fuente Única de la Verdad (*Single Source of Truth*) transaccional OLTP. Su motor MVCC, integridad referencial por claves foráneas, restricciones de dominio (`NUMERIC(10,2)`) y triggers de concurrencia (`fn_validar_stock_pedido` con `FOR UPDATE`) aseguran la reserva atómica de stock y la inmutabilidad contractual de los precios de venta.

En paralelo, **MongoDB** actúa como almacén de lectura optimizado y proyección de agregados (*Read Model* en patrón CQRS), gobernado por un validador **`$jsonSchema`** estricto y tipos **`Decimal128`** (`NumberDecimal`) para preservar la exactitud contable. Al almacenar los pedidos completos como documentos autosuficientes con sus líneas y datos del cliente embebidos, se maximiza la **localidad espacial de referencia**, permitiendo al frontend web y a logística recuperar comprobantes completos mediante una sola lectura indexada (`EXPRESS_IXSCAN` , 1 documento examinado) sin ejecutar los 4 `JOIN` relacionales ni penalizar las tablas operativas de PostgreSQL.